



Building a Regression Model in PyTorch

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 13



Post

PyTorch library is for deep learning. Some applications of deep learning models are to solve regression or classification problems.

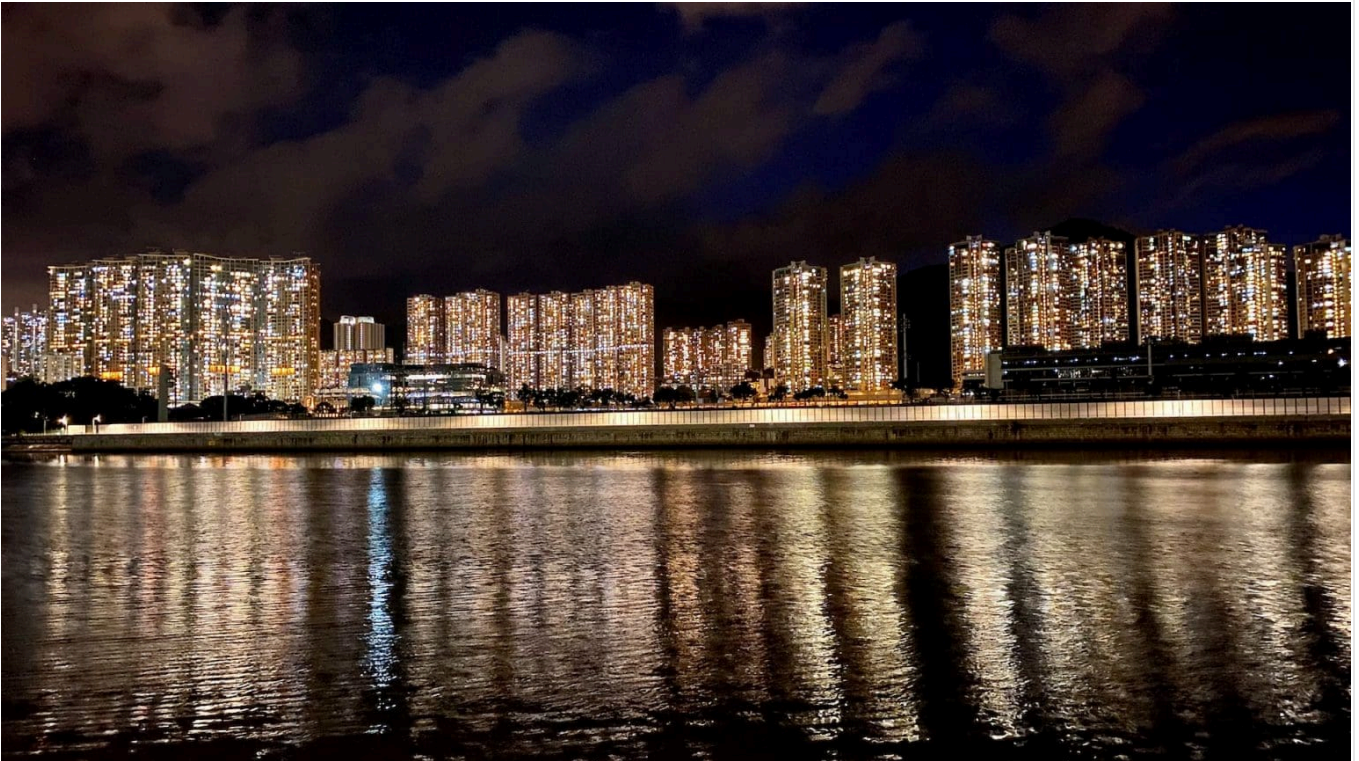
In this post, you will discover how to use PyTorch to develop and evaluate neural network models for regression problems.

After completing this post, you will know:

- How to load data from scikit-learn and adapt it for PyTorch models
- How to create a neural network for regression problem using PyTorch
- How to improve model performance with data preparation techniques

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Building a Regression Model in PyTorchPhoto by [Sam Deng](#). Some rights reserved.

Description of the Dataset

The dataset you will use in this tutorial is the [California housing dataset](#).

This is a dataset that describes the median house value for California districts. Each data sample is a census block group. The target variable is the median house value in USD 100,000 in 1990 and there are 8 input features, each describing something about the house. They are, namely,

- MedInc: median income in block group
- HouseAge: median house age in block group
- AveRooms: average number of rooms per household
- AveBedrms: average number of bedrooms per household
- Population: block group population
- AveOccup: average number of household members
- Latitude: block group centroid latitude
- Longitude: block group centroid longitude

This data is special because the input data is in vastly different scale. For example, the number of rooms per house is usually small but the population per block group is usually large. Moreover, most features should be positive but the longitude must be negative (because that's about California). Handling such diversity of data is a challenge to some machine learning models.

You can get the dataset from scikit-learn, which in turn, is downloaded from the Internet at realtime:

```
1 from sklearn.datasets import fetch_california_housing
2
3 data = fetch_california_housing()
4 print(data.feature_names)
5
6 X, y = data.data, data.target
```

Building a Model and Train

This is a regression problem. Unlike classification problems, the output variable is a continuous value. In case of neural networks, you usually use linear activation at the output layer (i.e., no activation) such that the output range theoretically can be anything from negative infinity to positive infinity.

Also for regression problems, you should never expect the model to predict the values perfectly. Therefore, you should care about how close the prediction is to the actual value. The loss metric that you can use for this is the mean square error (MSE) or mean absolute error (MAE). But you may also be interested in the root mean squared error (RMSE) because that's a metric in the same unit as your output variable.

Let's try the traditional design of a neural network, namely, the pyramid structure. A pyramid structure is to have the number of neurons in each layer decreasing as the network progresses to the output. The number of input features is fixed, but you set a large number of neurons on the first hidden layer and gradually reduce the number in the subsequent layers. Because you have only one target in this dataset, the final layer should output only one value.

One design is as follows:

```
1 import torch.nn as nn
2
3 # Define the model
4 model = nn.Sequential(
5     nn.Linear(8, 24),
6     nn.ReLU(),
7     nn.Linear(24, 12),
8     nn.ReLU(),
9     nn.Linear(12, 6),
10    nn.ReLU(),
11    nn.Linear(6, 1)
12 )
```

To train this network, you need to define a loss function. MSE is a reasonable choice. You also need an optimizer, such as Adam.

```
1 import torch.nn as nn
2 import torch.optim as optim
3
4 # loss function and optimizer
5 loss_fn = nn.MSELoss() # mean square error
6 optimizer = optim.Adam(model.parameters(), lr=0.0001)
```

To train this model, you can use your usual training loop. In order to obtain an evaluation score so you are confident that the model works, you need to split the data into training and test sets. You may also want to avoid overfitting by keeping track on the test set MSE. The following is the training loop with the train-test split:

```

1 import copy
2 import numpy as np
3 import torch
4 import tqdm
5 from sklearn.model_selection import train_test_split
6
7 # train-test split of the dataset
8 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
9 X_train = torch.tensor(X_train, dtype=torch.float32)
10 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
11 X_test = torch.tensor(X_test, dtype=torch.float32)
12 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
13
14 # training parameters
15 n_epochs = 100 # number of epochs to run
16 batch_size = 10 # size of each batch
17 batch_start = torch.arange(0, len(X_train), batch_size)
18
19 # Hold the best model
20 best_mse = np.inf # init to infinity
21 best_weights = None
22 history = []
23
24 # training loop
25 for epoch in range(n_epochs):
26     model.train()
27     with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
28         bar.set_description(f"Epoch {epoch}")
29         for start in bar:
30             # take a batch
31             X_batch = X_train[start:start+batch_size]
32             y_batch = y_train[start:start+batch_size]
33             # forward pass
34             y_pred = model(X_batch)
35             loss = loss_fn(y_pred, y_batch)
36             # backward pass
37             optimizer.zero_grad()
38             loss.backward()
39             # update weights
40             optimizer.step()
41             # print progress
42             bar.set_postfix(mse=float(loss))
43         # evaluate accuracy at end of each epoch
44         model.eval()
45         y_pred = model(X_test)
46         mse = loss_fn(y_pred, y_test)
47         mse = float(mse)
48         history.append(mse)
49         if mse < best_mse:
50             best_mse = mse
51             best_weights = copy.deepcopy(model.state_dict())
52
53 # restore model and return best accuracy
54 model.load_state_dict(best_weights)

```

In the training loop, `tqdm` is used to set up a progress bar and in each iteration step, MSE is calculated and reported. You can see how the MSE changed by setting the `tqdm` parameter `disable` above to `False`.

Note that in the training loop, each epoch is to run the forward and backward steps with the training set a few times to optimize the model weights, and at the end of the epoch, the model is evaluated using the test set. It is the MSE from the test set that is remembered in the list `history`. It is also the metric to evaluate a model, which the best one is stored in the variable `best_weights`.

After you run this, you will have the best model restored and the best MSE stored in the variable `best_mse`. Note that the mean square error is the average of the square of the difference between the predicted value and the actual value. The square root of it, RMSE, can be regarded as the average difference and it is numerically more useful.

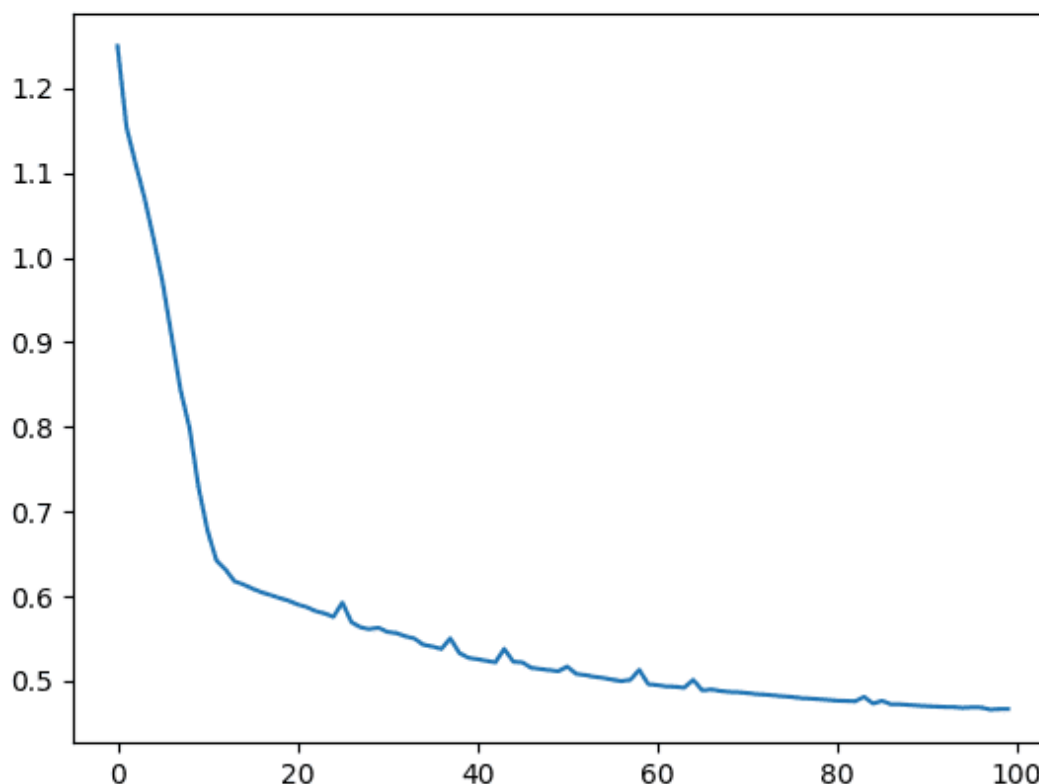
In below, you can show the MSE and RMSE, and plot the history of MSE. It should be decreasing with the epochs.

```
1 print("MSE: %.2f" % best_mse)
2 print("RMSE: %.2f" % np.sqrt(best_mse))
3 plt.plot(history)
4 plt.show()
```

This model produced:

```
1 MSE: 0.47
2 RMSE: 0.68
```

The MSE graph would like the following.



Putting everything together, the following is the complete code.

```

1  import copy
2
3  import matplotlib.pyplot as plt
4  import numpy as np
5  import pandas as pd
6  import torch
7  import torch.nn as nn
8  import torch.optim as optim
9  import tqdm
10 from sklearn.model_selection import train_test_split
11 from sklearn.datasets import fetch_california_housing
12
13 # Read data
14 data = fetch_california_housing()
15 X, y = data.data, data.target
16
17 # train-test split for model evaluation
18 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
19
20 # Convert to 2D PyTorch tensors
21 X_train = torch.tensor(X_train, dtype=torch.float32)
22 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
23 X_test = torch.tensor(X_test, dtype=torch.float32)
24 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
25
26 # Define the model
27 model = nn.Sequential(
28     nn.Linear(8, 24),

```



```

29     nn.ReLU(),
30     nn.Linear(24, 12),
31     nn.ReLU(),
32     nn.Linear(12, 6),
33     nn.ReLU(),
34     nn.Linear(6, 1)
35 )
36
37 # loss function and optimizer
38 loss_fn = nn.MSELoss() # mean square error
39 optimizer = optim.Adam(model.parameters(), lr=0.0001)
40
41 n_epochs = 100 # number of epochs to run
42 batch_size = 10 # size of each batch
43 batch_start = torch.arange(0, len(X_train), batch_size)
44
45 # Hold the best model
46 best_mse = np.inf # init to infinity
47 best_weights = None
48 history = []
49
50 for epoch in range(n_epochs):
51     model.train()
52     with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
53         bar.set_description(f"Epoch {epoch}")
54         for start in bar:
55             # take a batch
56             X_batch = X_train[start:start+batch_size]
57             y_batch = y_train[start:start+batch_size]
58             # forward pass
59             y_pred = model(X_batch)
60             loss = loss_fn(y_pred, y_batch)
61             # backward pass
62             optimizer.zero_grad()
63             loss.backward()
64             # update weights
65             optimizer.step()
66             # print progress
67             bar.set_postfix(mse=float(loss))
68         # evaluate accuracy at end of each epoch
69         model.eval()
70         y_pred = model(X_test)
71         mse = loss_fn(y_pred, y_test)
72         mse = float(mse)
73         history.append(mse)
74         if mse < best_mse:
75             best_mse = mse
76             best_weights = copy.deepcopy(model.state_dict())
77
78 # restore model and return best accuracy
79 model.load_state_dict(best_weights)
80 print("MSE: %.2f" % best_mse)
81 print("RMSE: %.2f" % np.sqrt(best_mse))
82 plt.plot(history)
83 plt.show()

```

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Improving the Model with Preprocessing

In the above, you see the RMSE is 0.68. Indeed, it is easy to improve the RMSE by polishing the data before training. The problem of this dataset is the diversity of the features: Some are with a narrow range and some are wide. And some are small but positive while some are very negative. This indeed is not very nice to most of the machine learning model.

One way to improve this is to apply a **standard scaler**. It is to convert each feature into their standard score. In other words, for each feature x , you replace it with

$$z = \frac{x - \bar{x}}{\sigma_x}$$

Where $\bar{x}$ is the mean of x and σ_x is the standard deviation. This way, every transformed feature is centered around 0 and in a narrow range that around 70% of the samples are between -1 to +1. This can help the machine learning model to converge.

You can apply the standard scaler from scikit-learn. The following is how you should modify the data preparation part of the above code:

```

1 import torch
2 from sklearn.model_selection import train_test_split
3 from sklearn.datasets import fetch_california_housing
4 from sklearn.preprocessing import StandardScaler
5
6 # Read data
7 data = fetch_california_housing()
8 X, y = data.data, data.target
9
10 # train-test split for model evaluation
11 X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=
12
13 # Standardizing data
14 scaler = StandardScaler()
15 scaler.fit(X_train_raw)
16 X_train = scaler.transform(X_train_raw)
17 X_test = scaler.transform(X_test_raw)
18
19 # Convert to 2D PyTorch tensors
20 X_train = torch.tensor(X_train, dtype=torch.float32)
21 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
22 X_test = torch.tensor(X_test, dtype=torch.float32)
23 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

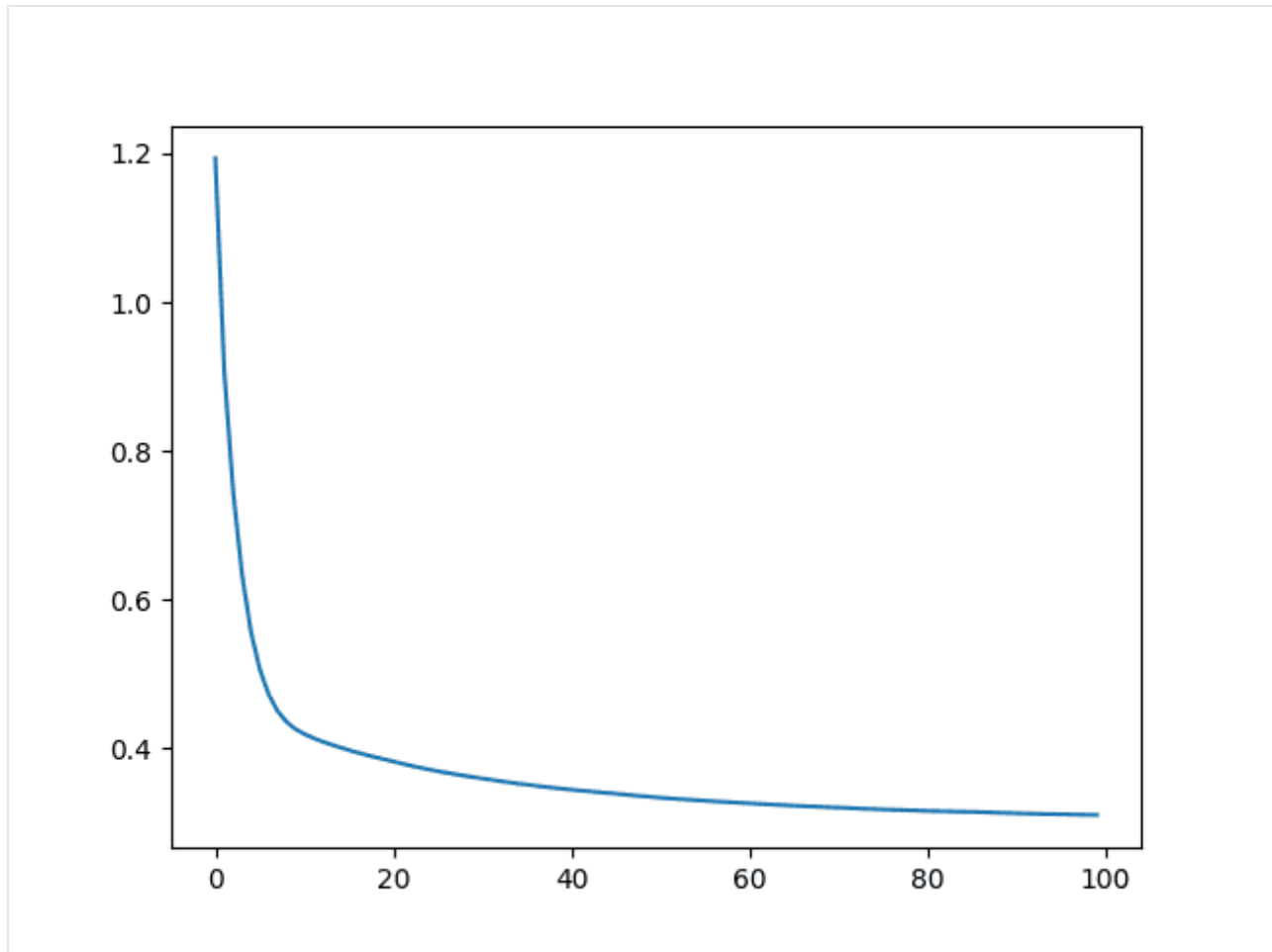
```


Note that standard scaler is applied after train-test split. The `StandardScaler` above is fitted on the training set but applied on both the training and test set. You must not apply the standard scaler to all data because nothing from the test set should be hinted to the model. Otherwise you are introducing **data leakage**.

Other than that, virtually nothing shall be changed: You still have 8 features (only they are not the same in value). You still use the same training loop. If you train the model with the scaled data, you should see the RMSE improved, e.g.,

```
1 MSE: 0.29
2 RMSE: 0.54
```

While the MSE history is in a similar falling shape, the y-axis shows it is indeed better after scaling:



However, you need to be careful at the end: When you use the trained model and apply to new data, you should apply the scaler to the input data before feed into the mode. That is, inference should be done as follows:

```
1 model.eval()
2 with torch.no_grad():
3     # Test out inference with 5 samples from the original test set
4     for i in range(5):
5         X_sample = X_test_raw[i: i+1]
6         X_sample = scaler.transform(X_sample)
7         X_sample = torch.tensor(X_sample, dtype=torch.float32)
```

```

8     y_pred = model(X_sample)
9     print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")

```

The following is the complete code:

```

1  import copy
2
3  import matplotlib.pyplot as plt
4  import numpy as np
5  import pandas as pd
6  import torch
7  import torch.nn as nn
8  import torch.optim as optim
9  import tqdm
10 from sklearn.model_selection import train_test_split
11 from sklearn.datasets import fetch_california_housing
12 from sklearn.preprocessing import StandardScaler
13
14 # Read data
15 data = fetch_california_housing()
16 X, y = data.data, data.target
17
18 # train-test split for model evaluation
19 X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle
20
21 # Standardizing data
22 scaler = StandardScaler()
23 scaler.fit(X_train_raw)
24 X_train = scaler.transform(X_train_raw)
25 X_test = scaler.transform(X_test_raw)
26
27 # Convert to 2D PyTorch tensors
28 X_train = torch.tensor(X_train, dtype=torch.float32)
29 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
30 X_test = torch.tensor(X_test, dtype=torch.float32)
31 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
32
33 # Define the model
34 model = nn.Sequential(
35     nn.Linear(8, 24),
36     nn.ReLU(),
37     nn.Linear(24, 12),
38     nn.ReLU(),
39     nn.Linear(12, 6),
40     nn.ReLU(),
41     nn.Linear(6, 1)
42 )
43
44 # loss function and optimizer
45 loss_fn = nn.MSELoss() # mean square error
46 optimizer = optim.Adam(model.parameters(), lr=0.0001)
47
48 n_epochs = 100 # number of epochs to run
49 batch_size = 10 # size of each batch
50 batch_start = torch.arange(0, len(X_train), batch_size)
51
52 # Hold the best model
53 best_mse = np.inf # init to infinity
54 best_weights = None
55 history = []
56
57 for epoch in range(n_epochs):
58     model.train()
59     with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:

```

```

60     bar.set_description(f"Epoch {epoch}")
61     for start in bar:
62         # take a batch
63         X_batch = X_train[start:start+batch_size]
64         y_batch = y_train[start:start+batch_size]
65         # forward pass
66         y_pred = model(X_batch)
67         loss = loss_fn(y_pred, y_batch)
68         # backward pass
69         optimizer.zero_grad()
70         loss.backward()
71         # update weights
72         optimizer.step()
73         # print progress
74         bar.set_postfix(mse=float(loss))
75     # evaluate accuracy at end of each epoch
76     model.eval()
77     y_pred = model(X_test)
78     mse = loss_fn(y_pred, y_test)
79     mse = float(mse)
80     history.append(mse)
81     if mse < best_mse:
82         best_mse = mse
83         best_weights = copy.deepcopy(model.state_dict())
84
85     # restore model and return best accuracy
86     model.load_state_dict(best_weights)
87     print("MSE: %.2f" % best_mse)
88     print("RMSE: %.2f" % np.sqrt(best_mse))
89     plt.plot(history)
90     plt.show()
91
92     model.eval()
93     with torch.no_grad():
94         # Test out inference with 5 samples
95         for i in range(5):
96             X_sample = X_test_raw[i:i+1]
97             X_sample = scaler.transform(X_sample)
98             X_sample = torch.tensor(X_sample, dtype=torch.float32)
99             y_pred = model(X_sample)
100            print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")

```

Of course, there is still room to improve the model. One way is to present the target in log scale or, equivalently, use mean absolute percentage error (MAPE) as the loss function. This is because the target variable is the value of houses and it is in a wide range. For the same error magnitude, it is more an issue for low-valued houses. It is your exercise to modify the above code to produce a better prediction.

Summary

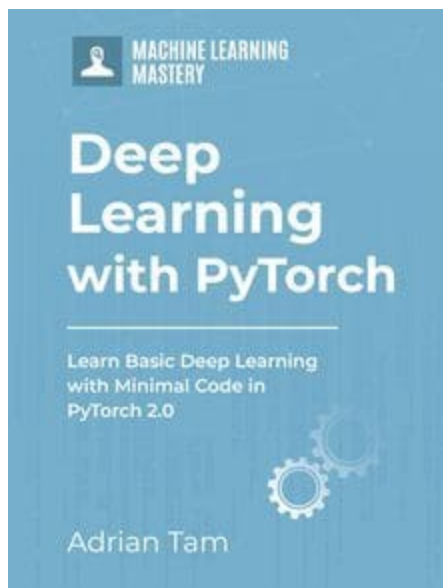
In this post, you discovered the use of PyTorch to build a regression model.

You learned how you can work through a regression problem step-by-step with PyTorch, specifically:

- How to load and prepare data for use in PyTorch
- How to create neural network models and choose a loss function for regression

- How to improve model accuracy by applying standard scaler

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

Share

Share

Post

More On This Topic



Building a Logistic Regression Classifier in PyTorch



Training a Linear Regression Model in PyTorch



Training a Single Output Multilinear Regression...



Training a Multi-Target Multilinear Regression Model...



Building a Multiclass Classification Model in PyTorch



Building a Binary Classification Model in PyTorch



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam →](#)

< [Building a Binary Classification Model in PyTorch](#) [Use PyTorch Deep Learning Models with scikit-learn](#) >

13 Responses to *Building a Regression Model in PyTorch*



George February 20, 2023 at 1:13 am #

REPLY ↩

Re Model Evaluation

Lines 92 and 935 respectfully are:

```
model.eval()
```

```
with torch.no_grad():
```

```
# Test out inference with 5 samples
```

```
for i in range(5):
```

And the validation evaluation excludes the code “with torch.no_grad():”

see line 76:

```
model.eval()
```

Why is `torch.no_grad()` used for one section for the evaluation and not the other?
The pytorch discussion website states that the code “`with torch.no_grad():`” can be used for both.



James Carmichael February 20, 2023 at 10:06 am #

REPLY ↩

Hi George...No particular reason or limitation. You should make the portions consistent and let us know what you find.



George February 23, 2023 at 2:12 am #

REPLY ↩

Hi James,

It's not how the program is run or when the evaluation is called. That won't tell me a lot of information.

What you need to know is what the pytorch documentation says rather than rely on how a particular model runs.



Arnold February 25, 2023 at 3:33 am #

REPLY ↩

Is there a way to turn off the intermediate output while iterating through the epochs?
Thanks.



Dimple May 26, 2023 at 5:32 pm #

REPLY ↩

Hello James,

I am writing the Regression Model in PyTorch for multiple inputs and multiple outputs, 1st I did the normalization (transformation) of inputs and outputs, but in the de-normalization stage, my code is showing the error, can you please help me to debug this?

`TypeError: 'int' object is not callable`



James Carmichael May 27, 2023 at 6:39 am #

REPLY ↩

Hi Dimple...While I have not encountered this particular error, the following resource may add clarity on how to address it:

<https://www.freecodecamp.org/news/typeerror-int-object-is-not-callable-how-to-fix-in-python/#:~:text=The%20%E2%80%9Cint%20object%20is%20not%20callable%E2%80%9D>

9D%20error%20occurs%20when%20you,while%20performing%20a%20mathematical%20operation.



Dimple June 14, 2023 at 6:01 pm #

REPLY ↩

Thanks James.

Could you please give one tutorial for PINNS problem in python using pytorch. I would really appreciate if you could show how to write custom loss functions using one or more outputs of a multi-output neural network.



Jorge Osorio June 1, 2023 at 5:43 am #

REPLY ↩

thank you very much for all your tutorials, they helped me a lot, you are great.



James Carmichael June 2, 2023 at 10:04 am #

REPLY ↩

You are very welcome Jorge! We greatly appreciate your support and feedback!



Nick October 20, 2023 at 1:24 pm #

REPLY ↩

Hi James,

Could you please clarify: what's the purpose of reshaping y_train and y_test tensors? (y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1))



James Carmichael October 21, 2023 at 9:18 am #

REPLY ↩

Hi Nick... The following resource may be of interest to you:

<https://discuss.pytorch.org/t/reshape-tensors-your-preserve-variable-information-structure/140300>



Nick October 27, 2023 at 2:05 am #

REPLY ↩

Thanks James.

I probably should specify my question. I can't figure out why should we use reshape here:


```
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
```

What's the sense of transforming `torch.Size([14447])` into `torch.Size([14447, 1])`? How does it help?



James Carmichael October 27, 2023 at 9:37 am #

REPLY ↩

Hi Nick...The following may be of interest to you:

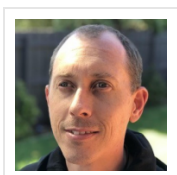
<https://dzone.com/articles/reshaping-pytorch-tensors>

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Building a Regression Model in PyTorch

Loving the Tutorials?

The Deep Learning with PyTorch EBook
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE

Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved
